

عنوان پروژه : Fintech
استاد درس : سرکار خانم صفوی
تهیه کنندگان : بنیامین ژیان و سروش عبقری

سامانه جامع پرداخت و مدیریت کیف پول الکترونیک (E-Wallet Core system)

کاربران پس از ثبت نام و احراز هویت (KYC)، وارد پنل خود می شوند. سیستم از طریق یک پرسشنامه آنلاین، میزان ریسک پذیری کاربر را سنجش می کند. سپس کاربر کیف پول خود را شارژ کرده و سیستم به صورت خودکار یا دستی، سبدهای پیشنهادی از دارایی ها (سهام، صندوق های درآمد ثابت و طلا) را به او پیشنهاد می کند. تراکنش ها به صورت آنی پردازش شده و نمودارهای سود و زیان به همراه پیش بینی های هوشمند در اختیار کاربر قرار می گیرد.

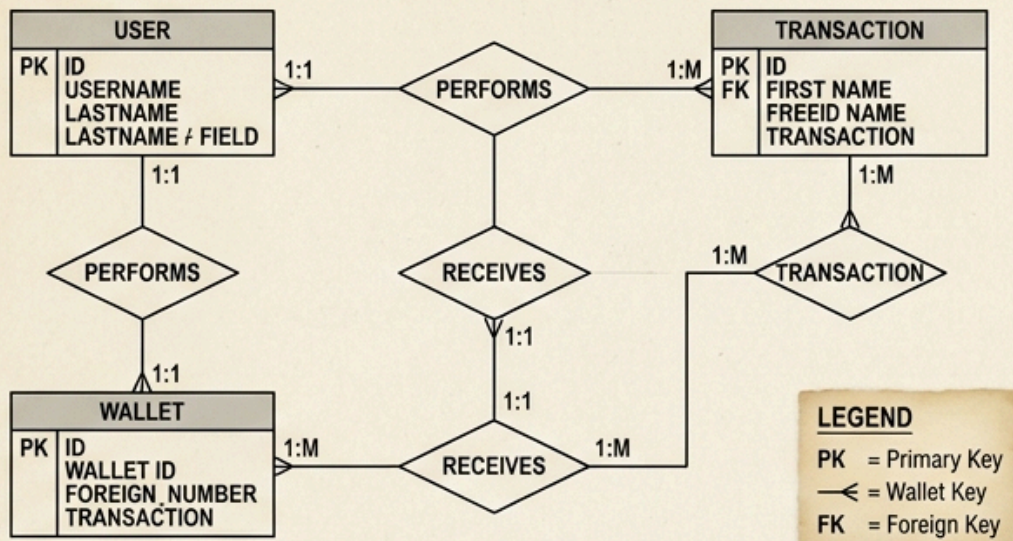
اهداف سیستم

- **هدف اصلی:** راحت کردن سرمایه گذاری و دسترسی آسان عموم مردم به سبد گردانی الگوریتمی.
- **کاهش ریسک معاملاتی:** استفاده از زیرسیستم مدیریت ریسک برای جلوگیری از کال مارجین شدن یا ضررهای سنگین.
- **اتوماسیون مالی:** حذف واسطه های انسانی در فرآیند تخصیص دارایی ها.

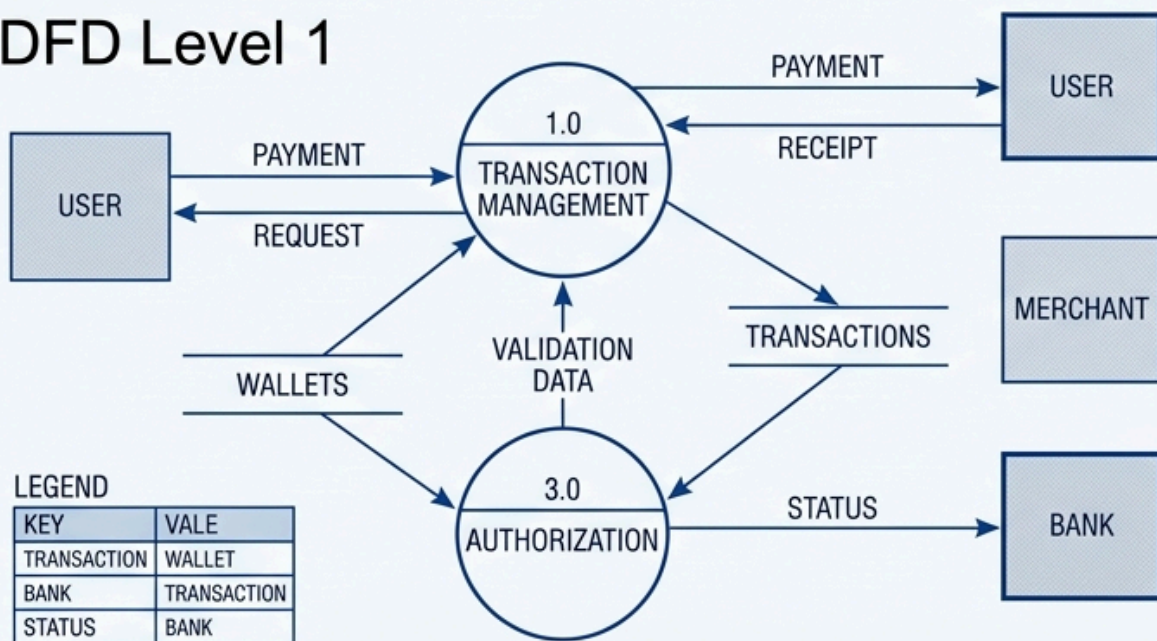
جامعه هدف

- خریداران آنلاین و کاربران اپلیکیشن های موبایلی.
- فروشگاه های اینترنتی و ارائه دهندگان خدمات آنلاین (Merchants).
- نهادهای ناظر و کارشناسان پشتیبانی و مالی شرکت.

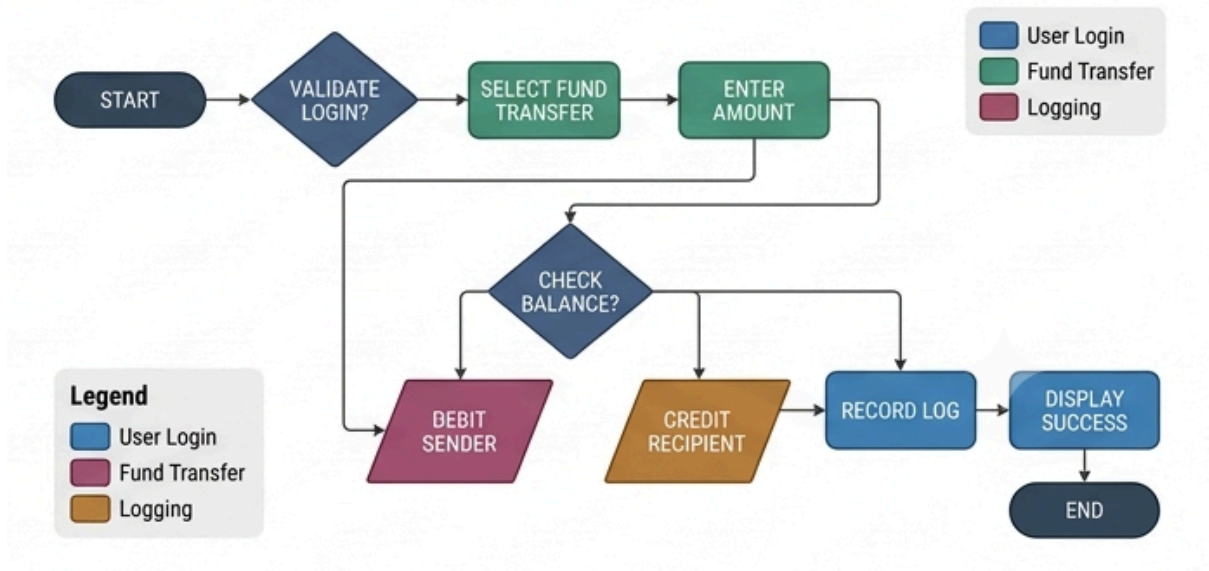
ER Diagram



DFD Level 1



Flowchart



کاربرد نمودارهای UML در پروژه

- نمودار Use Case : این نمودار مرز سیستم و تعاملات بازیگران (Actors) مانند کاربر و ادمین را با قابلیت‌های سیستم (مثل خرید دارایی، احراز هویت، مشاهده گزارشات) مشخص می‌کند.
- نمودار Class : ساختار شی‌گرا و کدهای برنامه را در لایه Backend مشخص می‌سازد. کلاس‌هایی مانند **Portfolio** و **Wallet Manager**، **Risk Analyzer** و **Builder** روابط ارث‌بری و وابستگی‌ها را تعیین می‌کنند.
- نمودار Sequence : زمان‌بندی و ترتیب ارسال پیام‌ها بین اشیاء مختلف در طول یک فرآیند (مانند فرآیند احراز هویت یا خرید) را به صورت خط زمانی نشان می‌دهد تا تداخل ایجاد نشود.

مدل معماری انتخاب شده و علت آن

معماری انتخابی: Microservices (میکروسرویس) مبتنی بر CQRS

برای این پروژه فین تک، از معماری میکروسرویس استفاده شده است که در آن بخش‌های مختلف مانند سرویس پردازش پرداخت (Payment)، سرویس تحلیل ریسک (Risk Evaluation) و سرویس گزارش‌دهی (Reporting) کاملاً از یکدیگر جدا هستند. همچنین الگوی CQRS در لایه داده اعمال شده است.

علت انتخاب:

1. مقیاس‌پذیری مستقل (Scalability): در زمان‌های شلوغی بازار، سرویس گزارش‌دهی و مشاهده قیمت‌ها بار پردازشی سنگینی دارد، در حالی که سرویس ثبت‌نام تراکنش خلوت‌تر است. با میکروسرویس می‌توان فقط منابع سرویس پرمصرف را ارتقا داد.
2. تاب‌آوری در برابر خطا (Fault Tolerance): اگر سرویس تولید نمودارها و تحلیل ریسک به هر دلیلی خطا کند، سرویس اصلی پرداخت و کیف پول به کار خود ادامه می‌دهد و کل سیستم دچار وقفه نمی‌شود.
3. امنیت بالا (Security): سرویس تراکنش‌های مالی در یک زون (zone) امنیتی مجزا قرار می‌گیرد و دسترسی سایر بخش‌های سیستم به آن محدود می‌شود.

بررسی سه خطای محتمل سیستم و روش‌های جلوگیری از آن‌ها

خطای اول: خطا در به‌روزرسانی موجودی (Race Condition)

- توضیح: Race Condition زمانی رخ می‌دهد که کاربر دو درخواست همزمان برای برداشت وجه ارسال کند و سیستم قبل از کسر درخواست اول، درخواست دوم را پردازش کند و موجودی منفی شود.
- روش جلوگیری: استفاده از مکانیزم Pessimistic Locking در سطح دیتابیس با استفاده از توزیع‌کننده‌های قفل مانند Redis Redlock. هنگام شروع تراکنش، ردیف مربوط به کیف پول کاربر قفل شده و تا پایان فرایند به هیچ درخواست همزمان دیگری پاسخ داده نمی‌شود.

خطای دوم: قطعی درگاه پرداخت در حین تراکنش (Third-Party API Downtime)

- توضیح: کاربر پول را از بانک واریز کرده اما هنگام بازگشت به سایت، ارتباط قطع می‌شود؛ در نتیجه پول از حساب کاربر کسر شده ولی کیف پول او شارژ نمی‌شود.

- روش جلوگیری: پیاده‌سازی الگوهای Idempotency Key و فرآیندهای Reconciliation (مغایرت‌گیری خودکار). سیستم به صورت دوره‌ای (مثلاً هر ۵ دقیقه) با اجرای یک کرون‌جاب (Cron Job) تراکنش‌های معلق را با API بانک تطبیق داده و در صورت تایید، کیف پول را به صورت خودکار شارژ می‌کند.

خطای سوم: درخواست‌های مخرب انبوه (DDoS Attacks) روی هسته معاملاتی

- توضیح: هکرها با ارسال میلیون‌ها درخواست فیک به سمت سرویس قیمت‌دهی یا خرید، باعث مصرف ۱۰۰ درصدی CPU و از دسترس خارج شدن سایت برای کاربران واقعی می‌شوند.
- روش جلوگیری: استفاده از فایروال‌های ابری (مانند Cloudflare) و اعمال سیاست‌های سخت‌گیرانه Rate Limiting (محدود سازی تعداد درخواست در ثانیه برای هر IP/کاربر) با استفاده از الگوریتم Token Bucket است .